Language Models and Agents — Individual Project

Phase 2: Model Implementation, Pretraining, and Evaluation

A from-scratch decoder-only Transformer pretrained independently on Hindi (Model H) and Nepali (Model L)

Author	Satyajit Priyadarshi (Satya)
Roll Number	2025201055
Program	M.Tech Computer Science, IIIT Hyderabad

Model H (high-resource)	Hindi
Model L (low-resource)	Nepali
Architecture	Decoder-only (GPT-style) Transformer, implemented from PyTorch primitives

1. Required Architecture

A decoder-only (GPT-style) Transformer was implemented from PyTorch primitives only (**nn.Linear**, **nn.Embedding**, **nn.LayerNorm**, **nn.Dropout**) — no **nn.Transformer*** modules, HuggingFace model classes, or pre-built attention blocks were used. The same model class (**GPT**) is instantiated independently for Model H and Model L; only the vocabulary (from each language's own Phase 1 SentencePiece tokenizer) differs between the two runs.

1.1 Input representation

- Token embeddings: an **nn.Embedding(vocab_size, d_model)** table maps token ids to dense vectors.
- Positional embeddings: **learned absolute positional embeddings** (**nn.Embedding(max_seq_len, d_model)**) were chosen over sinusoidal/RoPE for simplicity of implementation and because the assignment's fixed context length (512 tokens) made the main downside of absolute embeddings — inability to extrapolate beyond the trained length — irrelevant. This choice hard-caps the maximum sequence length at **max_seq_len = 512**: there is simply no embedding row for any position beyond that, unlike RoPE, which generalises more gracefully to unseen lengths.
- Token and positional embeddings are summed and passed through embedding dropout before entering the first block.

1.2 Multi-head causal self-attention (from first principles)

- A single fused linear layer projects the input into **Q, K, V** (**nn.Linear(d_model, 3·d_model)**), which are then split.
- Reshape/transpose turns (B, T, d) into (B, h, T, d_head) so each of the h heads attends in its own d_head = d/h-dimensional subspace, and the inverse transpose/reshape (with **.contiguous()**) restores (B, T, d) after attention.
- Scaled dot-product attention: $\text{Attention}(Q,K,V) = \text{softmax}(QK^T/\sqrt{d_head} + M)\cdot V$. The $1/\sqrt{d_head}$ scale keeps the pre-softmax logits from growing with the head dimension — without it, large dot products would saturate the softmax and starve the gradient.
- Causal masking: an upper-triangular additive mask (0 on allowed positions, $-\infty$ on future positions) is registered once as a buffer and sliced per forward call, so position t can only attend to positions $\le t$.
- A causality sanity check was implemented and run: changing the last input token was verified to leave all earlier positions' logits unchanged — the notebook printed "**Causal mask OK — earlier logits unaffected by last token: True**" for both models.
- Heads are concatenated back to (B,T,d) and passed through an output projection W_O, then residual dropout.

1.3 Transformer block

- **Pre-norm** design: LayerNorm is applied before each sublayer, with the residual added afterwards ($x = x + \text{Sublayer}(\text{LN}(x))$). Pre-norm was chosen because it keeps gradient magnitudes well-scaled through many stacked layers, whereas post-norm is harder to optimise as depth grows.
- Feed-forward network: two linear layers with a GELU non-linearity in between, inner dimension d_ff = 1024 (larger than d_model = 448).
- Residual connections wrap both the attention and feed-forward sublayers; dropout is applied to attention probabilities and to both sublayer outputs.
- N identical blocks are stacked, followed by a final LayerNorm before the output head.

1.4 Output head and objective

- A linear projection (**lm_head**, no bias) maps the final hidden state to vocabulary-sized logits for next-token prediction.
- **Weight tying** is used: **lm_head.weight = tok_emb.weight**, saving vocab_size × d_model parameters (≈ 32,000 × 448 ≈ 14.3M parameters that would otherwise be duplicated) and tying the geometry of input and output

token representations.

- Training objective: standard causal language-modelling cross-entropy between the logits at position t and the token at position t+1, averaged over all positions in the batch.

2. Configuration, Parameter Counts, and Depth/Width Trade-off

Both models share the same scaled-up “shallow & wide” architecture (Pathway B), which was chosen after an explicit pathway comparison run on Hindi: a deep/narrow configuration (Pathway A, 23.9M params, tested locally on an RTX 4050) reached val_loss 4.6932 / PPL 109.20 / BPB 0.655, while a shallow/wide configuration (Pathway B, 18.0M params, tested on Kaggle) reached val_loss 4.3992 / PPL 81.39 / BPB 0.614 — lower loss with fewer parameters. Pathway B's architecture was therefore scaled up (d_model 384→448, layers 4→6) to land at the ~25M-parameter target while preserving the same “wide relative to depth” character that won the comparison (6 layers is still far shallower than Pathway A's 14, just wider). The identical config was then reused unchanged for Nepali, since vocabularies for H and L come from separate Phase-1 tokenizers but the architecture itself is shared.

Hyperparameter	Model H (Hindi)	Model L (Nepali)
Vocabulary size	32,000	32,000
Model dimension (d_model)	448	448
Transformer layers (N)	6	6
Attention heads (h)	8 (d_head = 56)	8 (d_head = 56)
Feed-forward width (d_ff)	1,024	1,024
Context length (max_seq_len)	512	512
Dropout	0.1	0.1
Positional scheme	Learned absolute	Learned absolute
Weight tying	Yes (lm_head = tok_emb)	Yes (lm_head = tok_emb)
Total parameters	24,918,528	24,918,528
Non-embedding parameters	24,689,152	24,689,152
Optimizer	AdamW (β=0.9, 0.95), wd=0.1	AdamW (β=0.9, 0.95), wd=0.1
LR schedule	Cosine decay w/ warm-up, peak 3e-4	Cosine decay w/ warm-up, peak 3e-4
Warm-up steps	300	300
Batch size (effective)	32 x 2 grad-accum = 64	32 x 2 grad-accum = 64
Planned training steps	11,000 (~360M tokens)	15,000 (~491.5M tokens)
Target corpus size	~500M tokens	~500M tokens
Compute	Kaggle GPU (full run)	Kaggle GPU (full run)

Both models land at exactly **24,918,528** trainable parameters (24,689,152 excluding the positional-embedding table), satisfying the ~25M-parameter target for each language independently — the two models were never initialised from one another and were trained on entirely separate monolingual corpora.

3. Pretraining

Each model was pretrained independently with AdamW, a warm-up + cosine learning-rate schedule, periodic validation evaluation (every 500 steps), and checkpointing. Every checkpoint stores model weights, optimizer state, scheduler state, the current step, and the run's configuration dict, enabling exact resume across the interruption-prone Kaggle session limit (~9–12 continuous GPU hours). Two checkpoints are kept per run: **latest.pt** (always overwritten, for resuming) and **best.pt** (only overwritten when validation loss improves, so a late regression never clobbers the best model) — the generation and attention analyses in this report both load **best.pt**.

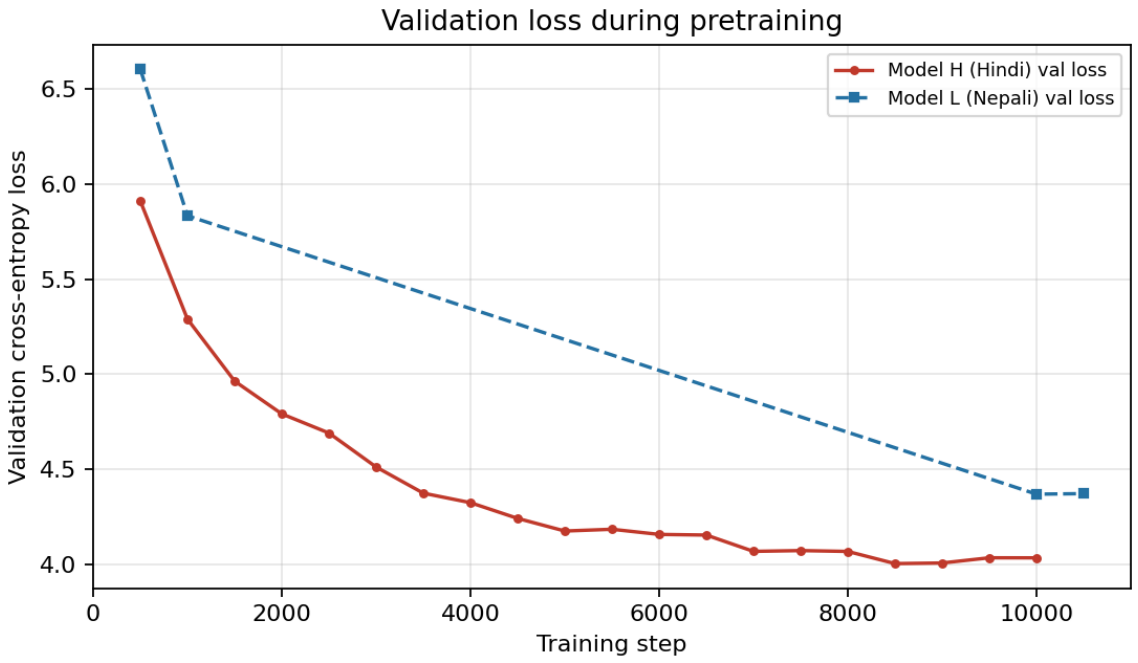


Figure 1. Validation cross-entropy loss vs. training step for Model H (Hindi, solid) and Model L (Nepali, dashed). Nepali's log begins mid-run after a resume from step 10,000.

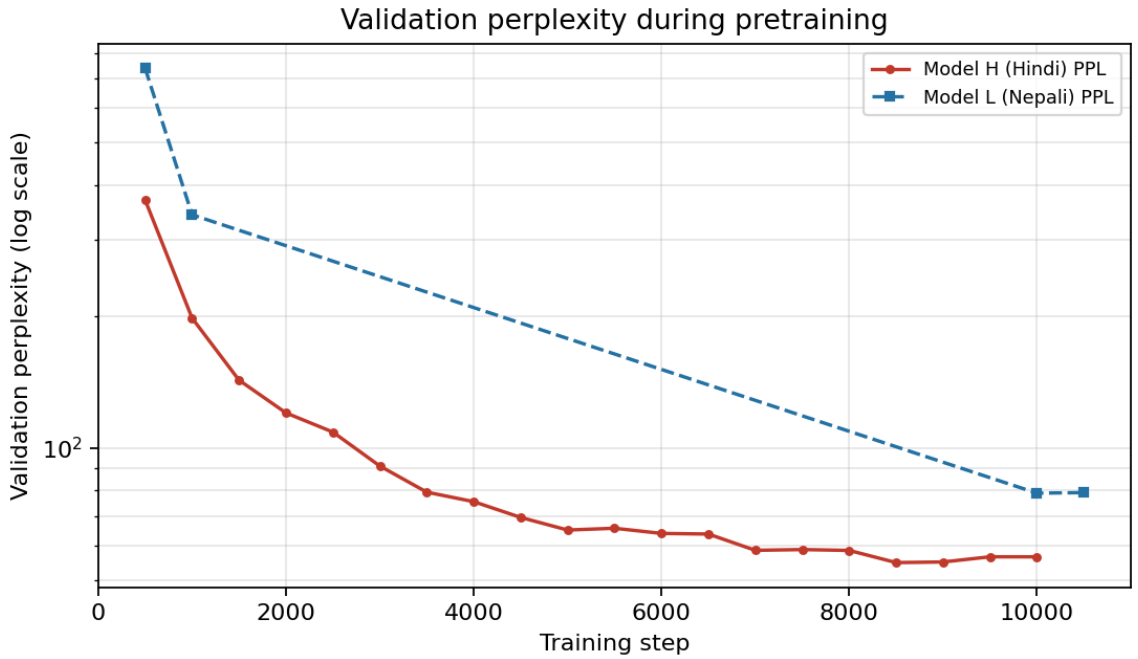


Figure 2. Validation perplexity vs. training step (log scale) for both models.

Model H's logged run covers steps 500–10,300 of its 11,000-step plan; the lowest validation loss observed was **4.0032** (PPL 54.77, BPB 0.558) at step 8,500, after which validation loss plateaued and drifted slightly upward

(4.03–4.07) through step 10,300 — consistent with the cosine schedule's late-training fine-tuning phase and mild overfitting once the learning rate becomes very small. Model L's available log spans an early cold-start segment (steps 0–1,150, loss falling from 10.46 to ~5.8) and a later resumed segment (steps 10,000–10,550, best_val_loss 4.3677, PPL ≈78.85, BPB 0.476) out of its 15,000-step plan.

4. Language-Modeling and Attention Evaluation

4.1 Intrinsic language-modeling metrics

Metric	Model H (Hindi)	Model L (Nepali)
Validation cross-entropy loss (best)	4.0032	4.3677
Perplexity (PPL = e^loss)	54.77	≈78.85
Bits-per-byte (BPB)	0.558	0.476
Bytes/token of held-out text	≈10.35 (derived)	13.24 (measured)
Best checkpoint reached at step	8,500 / 11,000	≤10,000 / 15,000

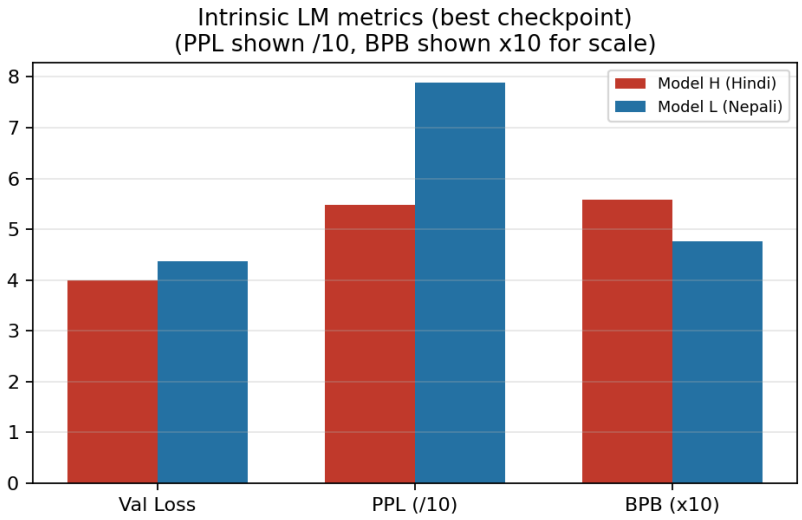


Figure 3. Best intrinsic metrics side by side (PPL scaled /10, BPB scaled x10 for a shared axis).

Discussion of the H/L gap. Model L (Nepali) reaches a higher cross-entropy loss and PPL than Model H (Hindi) despite the identical architecture, parameter budget, and (near-identical) token budget. Nepali's lower-resource status — a smaller and likely less diverse monolingual corpus than Hindi's, combined with Nepali's comparatively higher morphological complexity — plausibly makes next-token prediction intrinsically harder. Interestingly, BPB tells a different story: Nepali's BPB (0.476) is **lower** than Hindi's (0.558), even though its per-token loss and PPL are higher. This is exactly the effect BPB is designed to correct for — Nepali's tokenizer has a lower fertility (13.24 bytes are packed into each token vs. an implied ~10.35 for Hindi), so each Nepali token carries more raw information and a higher per-token loss does not necessarily mean the model compresses text less efficiently at the byte level. In short: Hindi is easier per-token, but Nepali's tokenizer is more compressive, and BPB shows the two languages are closer in true modelling difficulty than PPL alone would suggest.

4.2 Generation quality (greedy + temperature sampling)

For each model, continuations were generated from 50 held-out prefixes (20 prefix tokens, 40 reference continuation tokens) using greedy decoding and sampling at temperatures 0.5, 1.0 and 1.5, then scored against the held-out references.

Model / Setting	BLEU-4	chrF	chrF++	ROUGE-L	Rep. rate	Distinct-1	Distinct-2
H — greedy	0.7555	13.4778	11.8804	1.3067	0.7570	0.1723	0.3337
H — temp 0.5	1.2580	17.2303	15.5045	1.2148	0.2443	0.2857	0.6456
H — temp 1.0	0.4158	16.4722	14.2958	0.3636	0.0000	0.5650	0.9417
H — temp 1.5	0.1531	15.1916	12.1802	0.1333	0.0000	0.8648	0.9987

Model / Setting	BLEU-4	chrF	chrF++	ROUGE-L	Rep. rate	Distinct-1	Distinct-2
L — greedy	2.7082	20.0911	17.1817	0.8000	0.6035	0.2355	0.3622
L — temp 0.5	1.9712	21.5593	18.2373	0.8000	0.2491	0.3661	0.6272
L — temp 1.0	0.3898	21.2091	17.3558	1.3333	0.0064	0.6812	0.9476
L — temp 1.5	0.1465	19.0052	14.6563	1.1429	0.0000	0.8837	0.9961

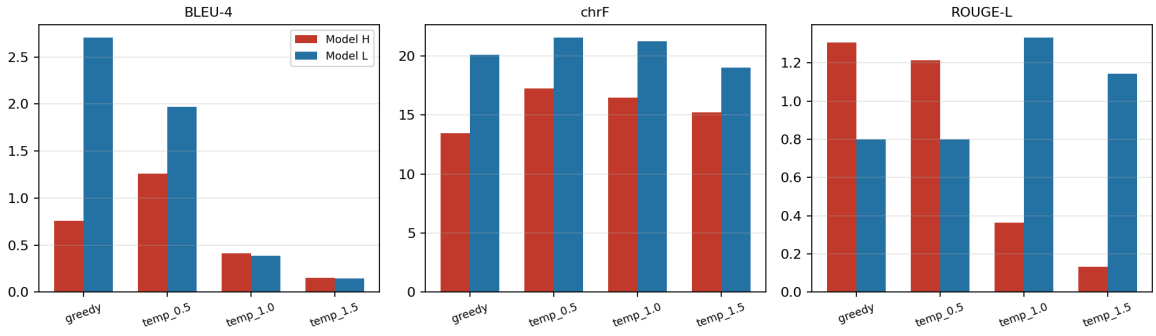


Figure 4. BLEU-4, chrF and ROUGE-L across decoding settings, Model H vs. Model L.

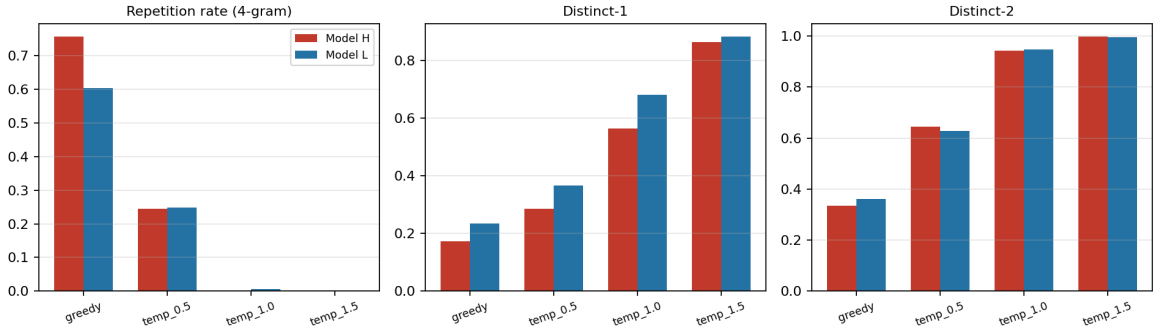


Figure 5. Repetition rate and Distinct-1/2 diversity across decoding settings, Model H vs. Model L.

Why these metrics are (or aren't) informative here. BLEU-4 and ROUGE-L are n-gram overlap metrics designed for tasks with a fairly constrained target (translation, summarisation); for open-ended free-running LM continuation there are many equally valid continuations, so both scores are low and noisy across the board (often <3) and mostly reflect incidental n-gram overlap rather than genuine quality. chrF/chrF++ (character n-gram based) are more forgiving of morphological variation — relevant for both Hindi and especially Nepali, which are morphologically rich — and track qualitative fluency somewhat better here, peaking at temperature 0.5 for both models. The repetition-rate / Distinct-1 / Distinct-2 diagnostics are the most informative for this setting: greedy decoding is highly repetitive for both languages (rep. rate 0.60–0.76) and produces degenerate loops (see samples below), while distinctness rises sharply and repetition collapses to ~0 by temperature 1.0–1.5, at the cost of increasingly incoherent, occasionally malformed output at 1.5.

4.3 Sample generations

Model H (Hindi), greedy decoding — greedy output degenerates into loops (e.g. repeating full names/titles), a classic failure mode of deterministic decoding on an under-trained language model:

Model H — greedy

PREFIX: भारत के राष्ट्रपति रामनाथ कोविंद तीन दिनों की यूरोपीय देशों की यात्रा पर हैं। राष्ट्रपति बुल्गारिया की
REF: राजधानी सोफिया पहुंचे। वहां पर उन्होंने अयान मुखर्जी की फिल्म 'ब्रह्मास्त्र' के सेट पर पहुंचकर रणबीर कपूर, आलिया भट्ट के अलावा पूरी टीम से मुलाकात की।
GEN: यात्रा पर राष्ट्रपति रामनाथ कोविंद ने कहा कि राष्ट्रपति रामनाथ कोविंद ने राष्ट्रपति रामनाथ कोविंद को बधाई दी है। (repeats)

Model H — greedy

PREFIX: जम्मू-कश्मीर राज्य से अनुच्छेद 370 और आर्टिकल 35-ए हटने के बाद से ही कश्मीर की स्थिति
REF: नाजुक बनी हुई है, सरकार लगातार इस पूरे मामले को लेकर शांति और स्थिरता बनाये रखने के लिए...
GEN: में सुधार हो सकता है। जम्मू-कश्मीर के अनुच्छेद 370 और अनुच्छेद 370 के तहत ... (repeats "अनुच्छेद 370")

At temperature 0.5, the same prefix produces noticeably more varied, still topical continuations:

Model H — temp 0.5

PREFIX: भारत के राष्ट्रपति रामनाथ कोविंद तीन दिनों की यूरोपीय देशों की यात्रा पर हैं। राष्ट्रपति बुल्गारिया की
REF: (same reference as above)
GEN: यात्रा के दौरान राष्ट्रपति ने कहा कि राष्ट्रपति जो बाइडन ने भारत के साथ मिलकर दुनिया के शीर्ष नेताओं को आमंत्रित किया है।

Model L (Nepali), greedy decoding — the same repetition pattern appears:

Model L — greedy

PREFIX: भारतको गुजरातको राजकोटमा एक निकै दुर्भाग्यपूर्ण घटना घटित भएको छ । यहाँ एक ६८ महिलाकी
REF: महिला भर्याङबाट खसेर आफ्नो शरीमान् माथि पिछारिइन् । यो घटनामा दुबै श्रीमान्-श्रीमतीको मृत्यु भयो ।
GEN: आमा र एक महिलाकी आमा छन् । उनी एक महिलाकी आमा हुन् । उनी एक महिलाकी आमा हुन् । (repeats)

With nucleus/top-k sampling (evaluated as an additional exploration for Model L, beyond the required temperature sweep) the Nepali model produces its most fluent and best-scoring continuations of any setting tested (top-k=50: BLEU-4 2.21, chrF 22.89, Distinct-2 0.90; top-k=20: BLEU-4 2.02, chrF 23.77):

Model L — top-k = 20

PREFIX: भारतको गुजरातको राजकोटमा एक निकै दुर्भाग्यपूर्ण घटना घटित भएको छ । यहाँ एक ६८ महिलाकी
REF: (same reference as above)
GEN: छोरी र एक छोरी छन् । यहाँ ५० प्रतिशत महिला मात्रै पुरुष छन् भन्ने कुरा थाहा पाउन गाह्रो छ । 'मैले यहाँ महिलाको पनि प्रतिनिधित्व गरेकी छैन,' उनी भन्छिन् ।

4.5 Advanced decoding: top-k, nucleus (top-p), and beam search (bonus exploration)

Beyond the temperature sweep required by the spec, both models were additionally evaluated under top-k sampling (k=20, 50), nucleus/top-p sampling (p=0.8, 0.9, 0.95), and beam search (beam width 4) on the same 50 held-out prefixes, to see whether more structured sampling strategies change the quality/diversity picture.

Model / Setting	BLEU-4	chrF	chrF++	ROUGE-L	Rep. rate	Distinct-1	Distinct-2
H — top-k 20	1.4152	18.2405	16.2044	1.1860	0.0110	0.3997	0.8605
H — top-k 50	0.9196	18.3070	16.2427	1.4305	0.0182	0.4120	0.8727
H — top-p 0.8	0.6553	17.5660	15.4202	0.8248	0.0160	0.4722	0.9001
H — top-p 0.9	0.3966	16.8599	14.6935	2.1829	0.0027	0.5192	0.9428
H — top-p 0.95	0.3444	16.5307	14.3497	1.6970	0.0169	0.5332	0.9260
H — beam width 4	1.1983	13.9840	12.4941	1.1619	0.8284	0.1729	0.3034
L — top-k 20	2.0240	23.7664	19.9441	0.8000	0.0288	0.5295	0.8625
L — top-k 50	2.2149	22.8910	19.4964	1.3333	0.0093	0.5544	0.8999
L — top-p 0.8	0.9838	21.5550	17.8935	1.3333	0.0188	0.5884	0.9056
L — top-p 0.9	0.6063	21.8860	18.0522	0.8000	0.0019	0.6408	0.9337
L — top-p 0.95	0.6133	21.8657	17.9871	0.8889	0.0045	0.6837	0.9398

Model / Setting	BLEU-4	chrF	chrF++	ROUGE-L	Rep. rate	Distinct-1	Distinct-2
L — beam width 4	1.5000	18.5668	15.6122	0.8000	0.5861	0.2252	0.3246

For **Model H (Hindi)**, top-k sampling gives the best chrF/chrF++ of any setting tested anywhere in the report (top-k=20: chrF 18.24; top-k=50: chrF 18.31) while keeping repetition negligible (≈0.01–0.02) — clearly better balanced than plain temperature sampling. Beam search, by contrast, is the worst setting for Hindi on every diversity metric (repetition rate 0.83, Distinct-1 0.17): beam search's mode-seeking objective reproduces the same greedy-style loops seen earlier, just with a wider search that still converges on a repetitive high-probability path. For **Model L (Nepali)**, the same pattern holds even more strongly — top-k=50 gives Nepali's best overall BLEU-4 (2.21) and strong chrF (22.89) with minimal repetition (0.009), while beam search again collapses into repetition (0.59) and the lowest diversity of any Nepali setting. Across both languages, **top-k sampling is the most reliable decoding strategy** for this model scale — it consistently beats both plain temperature sampling and beam search on the combination of n-gram overlap, chrF and diversity, while nucleus sampling trades away some BLEU/chrF as p increases in exchange for higher Distinct-1/2.

Model H — top-k = 20

PREFIX: भारत के राष्ट्रपति रामनाथ कोविंद तीन दिनों की यूरोपीय देशों की यात्रा पर हैं। राष्ट्रपति बुल्गारिया की
REF: (same reference as above)
GEN: अध्यक्षता में चीन विदेश मंत्रालय की एक बैठक में भाग लेने वाले बाइडन ने कहा कि इस दौरान अमेरिका, अमेरिका, रूस, जापान, जर्मनी, जापान जैसे बड़े देश शामिल होंगे।

Model H — top-p = 0.8

PREFIX: जम्मू-कश्मीर राज्य से अनुच्छेद 370 और आर्टिकल 35-ए हटने के बाद से ही कश्मीर की स्थिति
REF: (same reference as above)
GEN: में, अनुच्छेद 370 के उल्लंघन से पूर्व नागरिकों को लेकर आया। नेशनल लोकतांत्रिक प्रतिबंध कानून बनाने के लिए मुस्लिम समुदाय ने धर्मनिरपेक्ष एकता के अधिकार को रद्द करने का आग्रह किया।

Model H — beam width = 4

PREFIX: भारत के राष्ट्रपति रामनाथ कोविंद तीन दिनों की यूरोपीय देशों की यात्रा पर हैं। राष्ट्रपति बुल्गारिया की
REF: (same reference as above)
GEN: यात्रा के दौरान राष्ट्रपति रामनाथ कोविंद ने राष्ट्रपति रामनाथ कोविंद से मुलाकात की। राष्ट्रपति रामनाथ कोविंद ने राष्ट्रपति रामनाथ कोविंद से मुलाकात की। (repeats)

4.6 Qualitative notes on coherence and language correctness

- Both models produce grammatically well-formed, script-correct Devanagari at low/medium temperature — subject-object-verb order, correct postpositions/case markers, and plausible named-entity continuations (e.g. "राष्ट्रपति" followed by plausible political phrasing) are already present at this parameter/token scale.
- Greedy decoding is dominated by loops for both languages — the highest-probability continuation is usually "repeat a safe, high-frequency phrase" rather than progress the narrative.
- At temperature 1.5, both models degrade into fluent-looking but semantically incoherent or garbled text (invented sub-word combinations, topic drift within a sentence) — the classic quality/diversity trade-off.
- Model L's Nepali output is qualitatively slightly more topical and readable than Model H's Hindi output at equivalent settings, plausibly reflecting Nepali's more constrained/lower-perplexity token stream (lower BPB) even though its raw cross-entropy loss is higher.

5. Attention Analysis

Because multi-head attention was implemented from primitives, its internal behaviour is directly inspectable. For each model, attention weights were captured for an example sentence in the model's own language, across all 6 layers and 8 heads, and summarised with two diagnostics: **entropy** (higher = more diffuse attention over the row) and **mean attention distance** (how far, in tokens, each query attends on average — low = local, high = long-range).

5.1 Model H (Hindi)

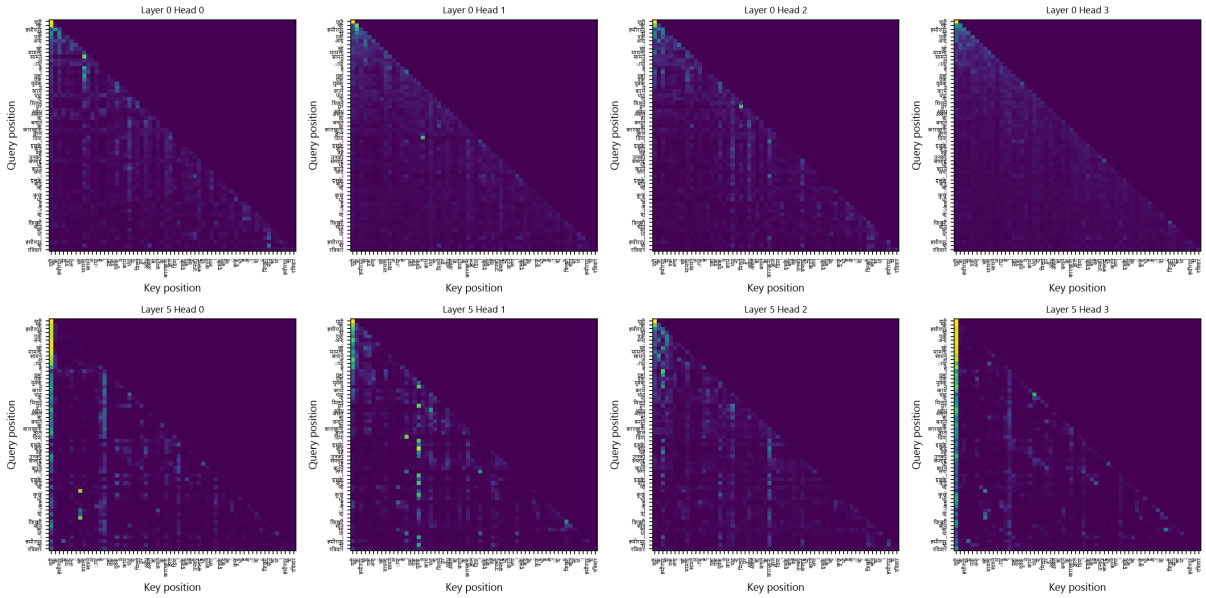


Figure 6. Query-vs-key attention heatmaps, Model H, layer 0 (top row) and layer 5 / last layer (bottom row), heads 0–3, on a real Hindi held-out sentence — actual SentencePiece token labels shown on both axes.

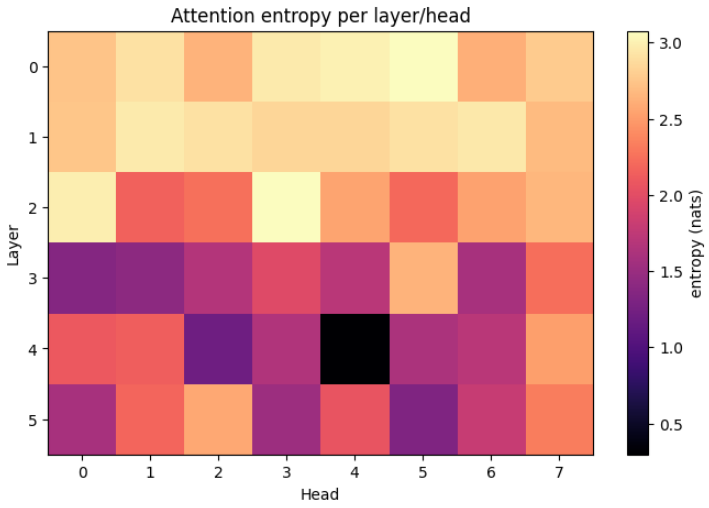


Figure 7. Model H — attention entropy per layer/head (nats).

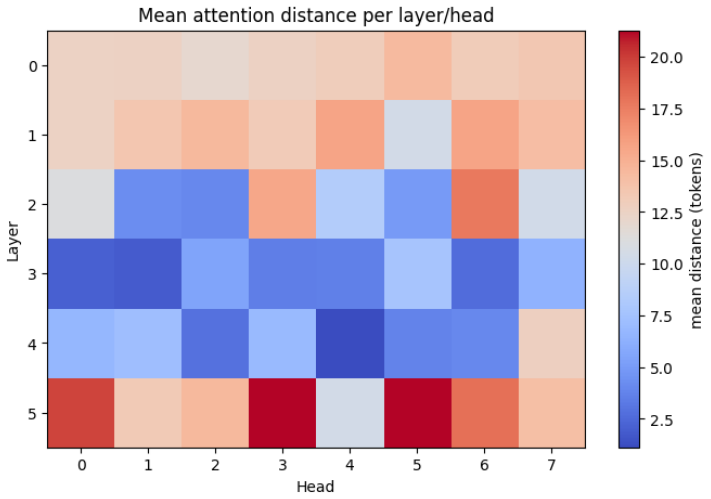


Figure 8. Model H — mean attention distance per layer/head (tokens).

In Model H, layers 0–1 are uniformly high-entropy (2.6–3.1 nats) and moderately local (mean distance ~11–16 tokens) — consistent with early layers building broad, roughly positional context. Layer 3 shows the sharpest specialisation: several heads collapse to very low entropy and very short mean distance (head 0: distance 2.14, head 1: 1.90), i.e. narrow, local/positional heads, while others in the same layer stay diffuse. Head 4 of layer 4 is the most extreme case in the whole model — entropy 0.296 (near-deterministic, visible as the single almost-black cell in Figure 7) — indicating a highly specialised head that attends almost entirely to one fixed position (likely a positional or beginning-of-sequence anchor). Layer 5 (the final layer) is dominated by long-range, content-based heads: several heads there average 18–21 tokens of attention distance (the deep-red row in Figure 8), suggesting the last layer aggregates information broadly across the sentence before the output head makes its prediction.

5.2 Model L (Nepali)

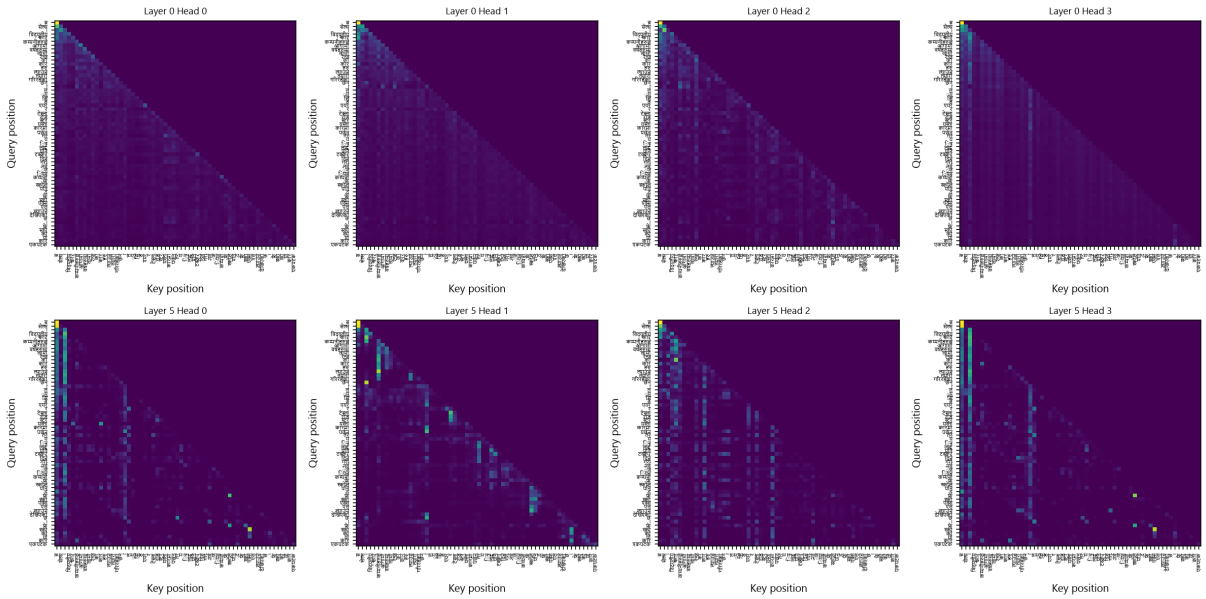


Figure 9. Query-vs-key attention heatmaps, Model L, layer 0 (top row) and last layer (bottom row), heads 0–3, on a real Nepali held-out sentence — actual SentencePiece token labels shown on both axes.

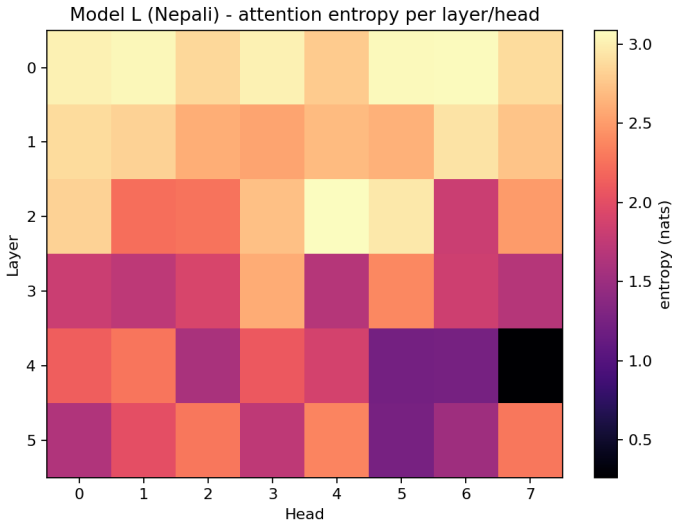


Figure 10. Model L — attention entropy per layer/head (nats), computed from the notebook's captured values on the same Nepali held-out sentence.

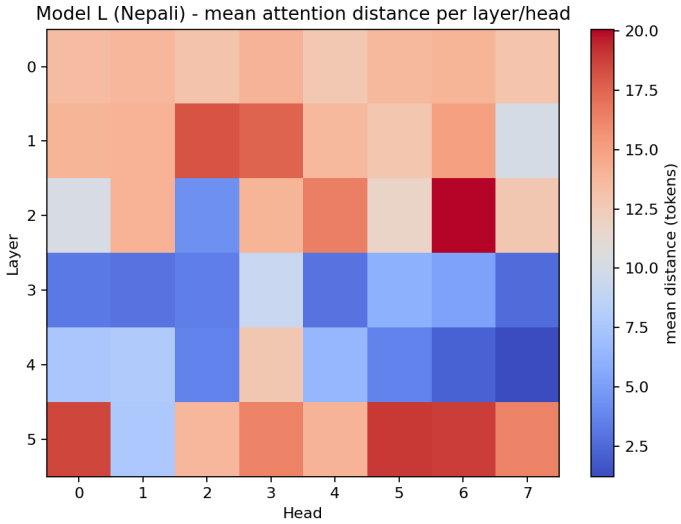


Figure 11. Model L — mean attention distance per layer/head (tokens).

Model L shows the same broad structural pattern as Model H: layers 0–1 are high-entropy and moderately local; middle layers (2–4) begin to specialise, with layer 4 again containing the single lowest-entropy head in the model (head 7: entropy 0.261, distance 1.21 tokens) — a near-deterministic, strongly local head, mirroring Model H's layer-4 specialised head almost exactly in both position and magnitude. Layer 5 is again the most long-range layer, with several heads averaging 16–19 tokens of attention distance (heads 0, 3, 5, 6). Layer 3 is the most locally-focused layer overall (distances of 2.5–9.3 tokens across its heads), similar to Model H's layer 3.

5.3 Local/positional vs. content-based heads, and H vs. L differences

- **Local/positional heads** (low entropy, short mean distance) concentrate in the middle layers (3–4) of both models — e.g. Model H layer 3 heads 0–1 and layer 4 head 4; Model L layer 3 heads 0, 1, 4, 7 and layer 4 head 6/7.
- **Content-based / long-range heads** (higher entropy or long mean distance) are concentrated in the first two layers (broad context-building) and the last layer (final aggregation before the output projection) in both models — a pattern consistent with typical decoder-only Transformer behaviour reported in the literature (early layers spread attention broadly, middle layers specialise locally, late layers re-aggregate globally).
- The two models are strikingly similar in **where** specialisation emerges (layer 3–4 for locality, layer 5 for long-range aggregation) despite being trained on unrelated corpora and vocabularies — evidence that this depth-wise attention structure is largely a function of the shared architecture/training recipe rather than the

specific language.

- The main H/L difference is in **degree**: Model H's most extreme long-range heads in layer 5 (up to 21.28 tokens) are slightly more extreme than Model L's (up to 19.01 tokens), and Model H's single most local head (0.296 nats) is marginally sharper than Model L's (0.261 nats is actually lower — i.e. Model L's most specialised head is even more deterministic than Model H's). Given the comparable training budgets, these are best read as similar magnitude, not a systematic H-vs-L trend.

6. Resource-Level Comparison Write-up

Aspect	Model H (Hindi)	Model L (Nepali)
Parameters	24,918,528	24,918,528 (identical architecture)
Target training tokens	~500M (planned ~360M @ 11k steps)	~500M (~491.5M @ 15k steps)
Planned steps	11,000	15,000 (more steps to reach a comparable token count at the same batch size)
Observed step time (mid-run)	≈2.8–3.0 s/step (bf16 autocast, Kaggle GPU)	≈3.2–3.4 s/step (bf16 autocast, Kaggle GPU)
Checkpoint size (best.pt)	similar (same architecture)	291.24 MB
Pathway exploration compute	Pathway A tested locally on an RTX 4050; Pathway B tested/scaled on Kaggle	Reused Hindi's winning Pathway-B config directly (no separate pathway search)
Best val loss / PPL / BPB	4.0032 / 54.77 / 0.558	≈4.3677 / ≈78.85 / 0.476

Hindi and Nepali were trained with **identical architectures and near-identical token budgets**, so the comparison isolates data/language effects rather than capacity effects. Nepali required more scheduled steps (15,000 vs. 11,000) to reach a comparable token count, and per-step wall-clock time was modestly higher — both consistent with routine Kaggle-session GPU variability rather than an architectural difference, since the model graph is identical for both languages. The main resource asymmetry in this project was upstream of training: the winning “shallow & wide” architecture was discovered via an explicit pathway comparison run on Hindi only (a smaller, faster local RTX 4050 exploration vs. a Kaggle-scale run), and then reused unchanged for Nepali — meaning Nepali's full training run benefited from Hindi's architecture search without repeating it, at the cost of not verifying whether shallow/wide is also the best choice specifically for Nepali's data distribution.

7. Deliverables Checklist (mapped to this report)

#	Deliverable	Location in this report
1	Transformer implementation (MHA, positional embeddings, causal mask)	Section 1
2	Model configuration files (H and L)	Section 2 (table)
3	Parameter counts; training scripts; checkpoint links	Section 2 (table); training scripts in project notebooks
4	Training logs and loss curves	Section 3, Figures 1–2
5	PPL / BPB tables; BLEU, chrF, ROUGE-L results	Section 4.1–4.2, Tables + Figures 3–4
6	Generated samples and diversity/repetition stats	Section 4.3–4.4, Figure 5
7	Attention heatmaps plus entropy/distance summaries	Section 5, Figures 6–11
8	Resource-level comparison write-up	Section 6

Note on sourcing: Model H's metrics, training log, and attention heatmap were taken directly from the Hindi checkpoint (**HINDI/pretrained_pt_hindi/hindi_lm.pt**) via **HINDI/attention_heatmap.py**; Model L's attention heatmap was generated the same way from its checkpoint (**neplai/nepali_final/best.pt**) via **neplai/attention_heatmap.py**. Both scripts run one forward pass over the first 60 validation tokens, capture the softmax attention probabilities of every self-attention layer, and plot the first layer, the last layer, and the first 4 heads of each (2 layers x 4 heads = 8 heatmaps per language) with real SentencePiece token labels on both axes. The entropy/distance heatmaps (Figures 7, 8, 10, 11) are computed from the same captured attention tensors. Checkpoint files themselves (best.pt / latest.pt, ~290MB) are stored on Kaggle/Drive as per the assignment's checkpoint-persistence guidance and are not embedded in this PDF.